

Audit one factor-graph program

Define, compute, test, and repair

DSAA 4011 | Fall 2026–27

Yuwen Huang | Fifty-minute laboratory

Preparation edition

What you will complete

By the end of this laboratory, you will be able to:

1. read a two-factor probabilistic model;
2. compute one conditional probability by hand;
3. specify a Python function that computes the same quantity;
4. design a test that exposes a plausible axis error; and
5. repair the code and explain the evidence.

The work fits one cycle

0–8 min

Understand and
compute

Define the model and obtain a small exact
answer.

8–18 min

Specify and test

Write the function contract and tests be-
fore using an AI coding agent.

18–42 min

Diagnose and re-
pair

Run the code, locate the first error, and
add a regression test.

42–50 min

Explain and sub-
mit

State the result, supporting evidence, and
remaining limitation.

Use the lecture example

Three neighboring machines have binary states. We observe Machine 3 and estimate Machine 2.

Variables. $x_i = 0$ means that Machine i is normal; $x_i = 1$ means that it is alert.

Observation. $x_3 = 1$.

Query. Compute $p(x_2 \mid x_3 = 1)$.

Define variables and assignments

Variable indices. Let $V = \{1, 2, 3\}$.

Variable domains. For each $i \in V$, $x_i \in \mathcal{X}_i = \{0, 1\}$.

Complete assignment. The vector $x = (x_1, x_2, x_3)$ lies in

$$\mathcal{X} = \mathcal{X}_1 \times \mathcal{X}_2 \times \mathcal{X}_3.$$

For example, $x = (0, 1, 1)$ means that Machines 2 and 3 are alert.

Read the two local factors

A **factor** is a nonnegative function of its declared variables. These factors give local weights:

$$f_{12} : \mathcal{X}_1 \times \mathcal{X}_2 \rightarrow \mathbb{R}_{\geq 0}$$

	$x_2 = 0$	$x_2 = 1$
$x_1 = 0$	4	1
$x_1 = 1$	2	5

$$f_{23} : \mathcal{X}_2 \times \mathcal{X}_3 \rightarrow \mathbb{R}_{\geq 0}$$

	$x_3 = 0$	$x_3 = 1$
$x_2 = 0$	3	1
$x_2 = 1$	1	4

For example, $f_{12}(1, 0) = 2$ and $f_{23}(0, 1) = 1$. A factor need not sum to one.

Draw the concrete factor graph



Circle: variable. **Square:** factor. **Edge:** the factor uses that variable.

The weight of one complete assignment is

$$w(x_1, x_2, x_3) = f_{12}(x_1, x_2) f_{23}(x_2, x_3).$$

The double circle marks the observed variable x_3 .

Read every edge

Here, $F = \{12, 23\}$ is the factor-index set. The argument sets are

$$S_{12} = \{1, 2\}, \quad S_{23} = \{2, 3\}.$$

Therefore, the graph has the edge set

$$E = \{(1, 12), (2, 12), (2, 23), (3, 23)\}.$$

Why is there no edge between x_1 and f_{23} ?

Write before discussion.

Formal definition from the example

For each $a \in F$, let $S_a \subseteq V$ be the indices of the variables used by factor f_a .

The **factor graph** is the bipartite graph

$$G = (V \cup F, E), \quad E = \{(i, a) : i \in S_a\}.$$

- Every edge joins a variable node to a factor node.
- The graph records the argument relation.
- The graph alone does not specify the arithmetic operations.

Declare the operations

In this probabilistic model, the weight of one assignment is

$$w(x) = \underbrace{\prod_{a \in F} f_a(x_{S_a})}_{\text{multiply local factor values}} .$$

If Q contains the retained variables and $H = V \setminus Q$ the eliminated variables, an unnormalized query is

$$g(x_Q) = \sum_{\underbrace{x_H}_{\text{add hidden assignments}}} w(x).$$

Multiply within one assignment; add across eliminated assignments.

State the question precisely

Observation. We observe $x_3 = 1$. Assignments with $x_3 = 0$ are no longer possible.

Query. Compute the two probabilities

$$p(x_2 = 0 \mid x_3 = 1) \quad \text{and} \quad p(x_2 = 1 \mid x_3 = 1).$$

Required output. A length-two vector ordered as $[p(x_2 = 0 \mid x_3 = 1), p(x_2 = 1 \mid x_3 = 1)]$.

Compute two weights

Let i denote a value of x_1 , j a value of x_2 , and k the observed value of x_3 .
Define

$$r_j = f_{23}(j, k) \sum_{i \in \{0,1\}} f_{12}(i, j).$$

For $k = 1$, compute r_0 and r_1 from the two tables.

Write before discussion.

Then normalize once

The normalization constant is $Z = r_0 + r_1$. The requested probabilities are

$$p(x_2 = j \mid x_3 = 1) = \frac{r_j}{Z}.$$

Normalize the two weights from your hand calculation. State one check on the result.

Write before discussion.

Keep a trusted reference

Write the exact vector from your hand calculation. What can one exact case establish, and what can it not establish?

Write before discussion.

Define normalization first

Function. `normalize(weights)`

Input. `weights` is a finite, nonnegative vector of shape $(d,)$ with a positive sum; d is the number of entries.

Return value. A vector of the same shape whose entries are

$$\text{weights}[j] / \sum_{t=0}^{d-1} \text{weights}[t].$$

Reject. A non-vector input, a negative or nonfinite entry, or a zero total.

Define the function first

Function. `chain_marginal(f12, f23, observed_x3)`

f12 A finite, nonnegative array of shape (d_1, d_2) ; entry $[i, j]$ equals $f_{12}(x_1 = i, x_2 = j)$.

f23 A finite, nonnegative array of shape (d_2, d_3) ; entry $[j, k]$ equals $f_{23}(x_2 = j, x_3 = k)$.

An integer k with $0 \leq k < d_3$.

observed_x3

Return value A finite, nonnegative vector of shape $(d_2,)$ that sums to one and represents $p(x_2 \mid x_3 = k)$.

Write tests before AI coding

Exact case

Record the vector from your hand calculation before running any code.

Shape $(d_2,)$; finite nonnegative values; sum equal to one.

Output contract

Include a zero-weight state with positive total weight.

Boundary case

Invalid input

Reject a negative entry, incompatible shapes, an out-of-range observation, or zero total weight.

Save this plan before asking AI to write code. A useful test exposes a specific failure.

Normalization can hide errors

The AI coding agent proposes:

```
1 def chain_marginal(f12, f23, observed_x3):  
2     raw = f12.sum(axis=1) * f23[:, observed_x3]  
3     return raw / raw.sum()
```

Predict the returned vector before running the program. Does normalization make it correct?

Write before discussion.

Axis choice changes the result

In f12, rows represent x_1 and columns represent x_2 . **Compute both axis sums. Which one removes x_1 and retains one value for each state of x_2 ?**

Write before discussion.

Use the exact case as a test

NumPy is imported as np; np.allclose compares two arrays within floating-point tolerance. expected = ... Enter your hand result.

```
actual = chain_marginal(...)
```

```
assert np.allclose(actual, expected)
```

Complete the test. Why is an asymmetric table useful here?

Write before discussion.

Make the smallest repair

Change only the line that violates the formula. Use variable names that state what each array represents.

Test more than one case

Regression test

Keep the exact asymmetric case. It prevents the repaired axis error from returning later.

Boundary test

Use one zero-weight state and verify that its returned probability is zero.

Invalid shape

Reject `f12.shape[1] != f23.shape[0]`; the two arrays then disagree about the number of x_2 states.

Invalid weights

Reject negative, nonfinite, or zero-total weights because they cannot define the requested probability.

Record a short evidence chain

Write one claim, the evidence that supports it, and one limitation of that evidence.

Write before discussion.

What to submit

Code and tests

Completed functions and at least one student-designed regression or boundary test.

The saved test plan and prediction.

Before AI use

Evidence

The smallest failing case, the minimal repair, and the passing result.

Audit note

A concise claim, supporting evidence, limitation, and only the agent interaction that changed a technical decision.

Do not submit a complete private chat history.

How feedback is organized

This formative laboratory uses the same four feedback headings as Coursework 1. It adds no separate mark.

Technical correctness

Is the declared calculation implemented correctly?

Reasoning and diagnosis

Does the test reveal the error and explain its cause?

Implementation and evidence

Do reproducible results support the claim?

Clarity and reproducibility

Can another reader follow and rerun the work?

Exit check

A program returns a nonnegative vector that sums to one. What additional evidence shows that it answers the declared query?

Write before discussion.

Carry forward. Week 2 replaces enumeration with reusable local messages while preserving the same model and query.